

REPORT DEL PROGETTO DI PROGRAMMAZIONE E STRUTTURE DATI A CURA DEGLI STUDENTI:

SANTILLO LUIGI NF12101147

SALERNO RAFFAELE NF12100527

CLASSE PEU-Z. TRACCIA NUMERO 2

1. Scelta degli ADT (Abstract Data Type) e motivazione teorica

Per la progettazione del sistema sono stati scelti diversi ADT (Abstract Data Type), cioè strutture dati astratte che definiscono il comportamento dei dati indipendentemente dalla loro implementazione concreta. La scelta di ogni struttura è stata effettuata considerando:

- a) efficienza delle operazioni principali;
- b) semplicità gestionale;
- c) tipo di accesso richiesto ai dati;
- d) frequenza di inserimenti, eliminazioni e ricerche.

2. Lista collegata → Studenti registrati

La gestione degli studenti registrati richiede una struttura dinamica, capace di crescere o diminuire durante l'esecuzione del programma senza dover riallocare continuamente memoria. Per questo motivo viene utilizzata una lista collegata semplice, composta da nodi dove ogni nodo contiene:

- a) il dato (es. nome, matricola, corso);
- b) un puntatore al nodo successivo.

3. Vantaggi

- a) Inserimento rapido di nuovi studenti.

- b) Nessun limite fisso di dimensione.
- c) gestione della memoria.
- d) Facilità di scansione sequenziale.

4. Svantaggi

- a) Accesso non diretto agli elementi.
- b) Ricerca più lenta rispetto ad array ordinati.

5. Complessità principali

Operazione	Complessità
a) Inserimento in testa	$O(1)$
b) Ricerca	$O(n)$
c) Eliminazione	$O(n)$

6. Motivazione della scelta

Gli studenti registrati vengono principalmente aggiunti, cercati e visualizzati sequenzialmente. Una lista collegata rappresenta quindi una soluzione semplice ed efficiente.

SECONDO ADT

1. Lista doppiamente collegata → Prenotazioni

Le prenotazioni richiedono maggiore flessibilità nella navigazione e nella cancellazione degli elementi. Per questo viene scelta una lista doppiamente collegata. Ogni nodo contiene:

- a) il dato;
- b) un puntatore al nodo successivo;
- c) un puntatore al nodo precedente.

2. Vantaggi

- a) Eliminazione veloce di una prenotazione, navigazione avanti/indietro.
- b) Maggiore controllo sulla struttura.

3. Svantaggi

- a) Maggiore utilizzo di memoria.
- b) Implementazione più complessa.

4. Complessità

a) Operazione	Complessità
b) Inserimento	$O(1)$
c) Eliminazione nodo noto	$O(1)$
d) Ricerca	$O(n)$

5. Motivazione della scelta

Nel sistema di prenotazione può essere necessario annullare prenotazioni, modificare l'ordine e scorrere le prenotazioni avanti e indietro. La lista doppiamente collegata rende queste operazioni più efficienti.

TERZO ADT UTILIZZATO

1. Coda (Queue) → Lista di attesa

Per la gestione della lista di attesa viene utilizzata una coda FIFO (First In First Out). La coda è una struttura dati lineare in cui il primo elemento inserito è il primo a essere rimosso. Le operazioni fondamentali sono:

- a) enqueue → inserimento in coda;
- b) dequeue → rimozione dalla testa.

2. Vantaggi

- a) Gestione ordinata delle richieste.
- b) Rispetta l'ordine cronologico.
- c) Implementazione semplice.

3. Complessità

Operazione	Complessità
a) Enqueue	$O(1)$
b) Dequeue	$O(1)$

4. Motivazione della scelta

La lista di attesa deve garantire equità: chi arriva prima viene servito prima. La coda FIFO rappresenta quindi la soluzione ideale.

QUARTO ADT UTILIZZATO

1. Albero Binario di Ricerca (BST) → Storico accessi

Per memorizzare e consultare rapidamente lo storico degli accessi viene utilizzato un Binary Search Tree (BST). Un BST è una struttura gerarchica in cui:

- a) ogni nodo possiede al massimo due figli;
- b) i valori minori si trovano a sinistra;
- c) i valori maggiori si trovano a destra.

Questo consente ricerche ordinate ed efficienti.

2. Vantaggi

- a) Ricerca veloce.
- b) Ordinamento naturale dei dati.
- c) Inserimenti dinamici.

3. Svantaggi

- a) Peggioramento delle prestazioni se l'albero è sbilanciato.

4. Complessità

Operazione	Caso medi	Caso peggiore
a) Ricerca	$O(\log n)$	$O(n)$
b) Inserimento	$O(\log n)$	$O(n)$
c) Eliminazione	$O(\log n)$	$O(n)$

5. Motivazione della scelta

Lo storico accessi richiede ricerca per matricola, ordinamento cronologico e consultazione rapida. Il BST permette di organizzare efficacemente i dati mantenendoli ordinati.

QUINTO ADT UTILIZZATO

1. Lista semplice → Studenti presenti

Per rappresentare gli studenti attualmente presenti viene utilizzata una struttura lineare semplice. Una lista semplice consente di:

- a) memorizzare elementi in sequenza;
- b) effettuare scansioni rapide;
- c) aggiungere o rimuovere studenti presenti.

2. Vantaggi

- a) Facilità di implementazione.
- b) Gestione intuitiva.
- c) Buona efficienza per piccoli insiemi di dati.

3. Motivazione della scelta

L'elenco degli studenti presenti viene aggiornato frequentemente e consultato in modo sequenziale. Una lista semplice risulta quindi adeguata e poco costosa dal punto di vista computazionale. Nel sistema progettato le liste collegate garantiscono flessibilità, la coda assicura gestione corretta delle attese, il BST migliora l'efficienza delle ricerche e le strutture lineari semplificano la gestione dei dati correnti. Questa combinazione permette di ottenere un sistema:

- a) efficiente;
- b) modulare;
- c) facilmente espandibile;
- d) ottimizzato nell'uso della memoria e nelle prestazioni.

PROGETTAZIONE

1. architettura del sistema

- a) student.h / student.c
- b) reservation.h / reservation.c
- c) Student_list.h / Student_list.c
- d) system.h / system.c
- e) main.c

2. Architettura del Sistema

Il sistema è progettato seguendo un'architettura modulare e astratta, suddividendo le responsabilità in più componenti indipendenti. Ogni modulo è composto da:

- a) un file .h → definisce l'interfaccia pubblica
- b) un file .c → implementa la logica interna

Questa separazione permette:

- a) maggiore manutenibilità

- b) riuso del codice
- c) incapsulamento dei dati
- d) riduzione dell'accoppiamento
- e) maggiore leggibilità

L'architettura segue i principi della programmazione procedurale in C con astrazione tramite ADT (Abstract Data Type). Struttura dei Moduli:

1. Modulo Student File

- a) student.h
- b) student.c

2. Responsabilità

Il modulo Student rappresenta l'entità logica "studente". Gestisce:

- a) dati anagrafici
- b) identificazione dello studente
- c) operazioni di accesso/modifica

Questo modulo implementa un ADT (Abstract Data Type). L'utente del modulo non deve conoscere la struttura interna dello studente, ma soltanto utilizzare le funzioni esposte nell'header.

Esempio concettuale

```
typedef struct {  
    int id;  
    char name[50];  
    char surname[50];  
} Student;
```

3. Operazioni tipiche

- a) creazione studente
- b) modifica dati
- c) stampa informazioni

d) confronto tra studenti

4. Vantaggi

- a) incapsulamento
- b) riutilizzabilità
- c) isolamento della logica anagrafica

SECONDO MODULO:

1. Modulo Reservation

2. File

- a) reservation.h
- b) reservation.c

3. Responsabilità

Gestisce il concetto di prenotazione. Una prenotazione collega uno studente, una risorsa/servizio, una data o fascia oraria. Il modulo rappresenta il livello di gestione delle relazioni tra entità. Introduce:

- a) gestione dello stato
- b) validazione dei dati
- c) controllo disponibilità

Può essere visto come il livello di business logic relativo alle prenotazioni.

4. Operazioni tipiche

- a) creare prenotazione
- b) annullare prenotazione
- c) verificare disponibilità
- d) stampare prenotazioni

TERZO MODULO

1. Modulo Student_List

2. File

- a) Student_list.h
- b) Student_list.c
- 3. Responsabilità

Gestisce collezioni di studenti. Il modulo implementa una struttura dati dedicata alla memorizzazione e gestione multipla degli utenti. Questo componente realizza un contenitore astratto. Può essere implementato tramite lista concatenata, array dinamico, coda o vettore. La scelta dipende dai requisiti prestazionali.

- 4. Operazioni tipiche
 - a) inserimento studente
 - b) eliminazione
 - c) ricerca
 - d) scansione lista

5. Complessità algoritmica

Operazione	Lista concatenata
a) Inserimento	$O(1)$
b) Ricerca	$O(n)$
c) Eliminazione	$O(n)$

- 6. Vantaggi modulari
 - a) separazione tra dati e gestione
 - b) possibilità di sostituire la struttura senza modificare gli altri moduli

QUARTO MODULO

- 1. Modulo System
- 2. File
 - a) system.h
 - b) system.c
- 3. Responsabilità

Coordina tutti i moduli del sistema. Rappresenta il livello alto dell'applicazione. Gestisce: il flusso operativo, le interazioni tra moduli, la logica globale e il controllo applicativo. Questo modulo segue il principio di: orchestrazione e controllo centralizzato. Agisce come facade layer, semplificando l'interazione tra i vari sottosistemi.

4. Funzioni tipiche

- a) inizializzazione sistema
- b) caricamento dati
- c) gestione menu
- d) coordinamento prenotazioni/studenti
- e) Benefici architetturali
- f) riduzione dipendenze dirette
- g) centralizzazione della logica
- h) migliore scalabilità

SPECIFICA SINTATTICA E SEMANTICA DELLE FUNZIONI UTILIZZATE

1. Modulo Student

1. student_create

Specifica Sintattica:

```
Student* student_create(int m, const char* nome, const char* corso);
```

Parametri

int m → matricola dello studente

const char* nome → stringa contenente il nome

const char* corso → stringa contenente il corso di laurea

Tipo di ritorno

Student*

Specifica Semantica

Alloca dinamicamente una struttura Student, inizializza i campi con i valori passati come argomenti e restituisce il puntatore alla nuova struttura.

Precondizioni

nome != NULL

corso != NULL

nome e corso devono essere stringhe terminate da '\0'

Le stringhe devono avere lunghezza compatibile con gli array:

`strlen(nome) < 50`

`strlen(corso) < 50`

Postcondizioni

Viene allocata memoria heap per una struttura Student

I campi della struttura contengono:

a) matricola = m

b) nome = nome

c) corso = corso

Valore di ritorno

Puntatore alla struttura Student creata

In caso di fallimento di malloc, il comportamento attuale del codice è indefinito perché non viene controllato NULL

Effetti Collaterali

Allocazione di memoria dinamica sull'heap

2. student_get_matricola

Specifica Sintattica

```
int student_get_matricola(const Student* s);
```

Parametri

const Student* s → puntatore allo studente

Tipo di ritorno: Int

Specifica Semantica

Restituisce la matricola dello studente.

Precondizioni

a) s != NULL

b) s deve puntare a una struttura valida

Postcondizioni

a) Nessuna modifica allo stato del programma

b) Valore di ritorno

c) Valore del campo matricola

Effetti Collaterali

Nessuno

3. student_get_nome

Specifica Sintattica

```
const char* student_get_nome(const Student* s);
```

Parametri

const Student* s

Tipo di ritorno: const char*

Specifica Semantica

Restituisce il nome dello studente.

Precondizioni

- a) $s \neq \text{NULL}$
- b) s valido

Postcondizioni

- a) Nessuna modifica dei dati
- b) Valore di ritorno
- c) Puntatore alla stringa nome

Effetti Collaterali

- a) Nessuno

4. `student_get_corso`

Specifica Sintattica

```
const char* student_get_corso(const Student* s);
```

Parametri

`const Student* s`

Tipo di ritorno: `const char*`

Specifica Semantica

Restituisce il corso di laurea dello studente.

Precondizioni

- b) $s \neq \text{NULL}$
- c) s valido

Postcondizioni

Nessuna modifica allo stato

Valore di ritorno

Puntatore alla stringa corso

Effetti Collaterali

Nessuno

5. student_save

Specifica Sintattica

```
void student_save(const Student* s, FILE* f);
```

Parametri

const Student* s → studente da salvare

FILE* f → file aperto in scrittura

Tipo di ritorno

void

Specifica Semantica

Scrive su file i dati dello studente nel formato:

matricola nome corso

Precondizioni

s != NULL

f != NULL

f aperto in modalità scrittura

Postcondizioni

I dati dello studente vengono scritti nel file

Valore di ritorno

Nessuno

Effetti Collaterali

Modifica il contenuto del file

Avanza il cursore del file

6. student_load

Specifica Sintattica

Student* student_load(FILE* f);

Parametri

FILE* f → file aperto in lettura

Tipo di ritorno

Student*

Specifica Semantica

Legge dal file matricola, nome e corso e crea dinamicamente un nuovo Student.

Precondizioni

f != NULL

f aperto in lettura

Il file deve contenere dati nel formato:

int stringa stringa

Postcondizioni

Se la lettura ha successo:

viene creato un nuovo Student

Se fallisce:

nessuna allocazione valida

Valore di ritorno

Puntatore al nuovo Student se la lettura riesce

NULL se fscanf non legge correttamente 3 elementi

Effetti Collaterali

Avanza il cursore del file

Allocazione dinamica di memoria

7. student_destroy

Specifica Sintattica

```
void student_destroy(Student* s);
```

Parametri

Student* s

Tipo di ritorno

void

Specifica Semantica

Libera la memoria dinamica occupata da uno Student.

Precondizioni

s deve essere stato allocato dinamicamente

s non deve essere già stato liberato

Postcondizioni

La memoria viene restituita al sistema

s diventa dangling pointer

Valore di ritorno

Nessuno

Effetti Collaterali

Liberazione della memoria heap

Possibile comportamento indefinito se usato dopo free

Reservation

Specifica delle funzioni

1. res_create

Specifica sintattica

```
Reservation* res_create(int m, const char* d, int f, int p);
```

Semantica

Crea dinamicamente una nuova prenotazione inizializzando tutti i campi della struttura.

Precondizioni

- `d != NULL`
- `strlen(d) < 20`
- memoria disponibile per malloc

Postcondizioni

- viene allocata una nuova struttura Reservation
- i campi vengono inizializzati:
- `matricola = m`

- data = d
- fascia = f
- posto = p
- checked = 0

Valore di ritorno

ValoreSignificatopuntatore a Reservationprenotazione creata correttamenteNULLallocazione fallita (non gestita esplicitamente nel codice)

2. res_get_matricola

Specifica sintattica

```
int res_get_matricola(const Reservation* r);
```

Semantica

Restituisce la matricola associata alla prenotazione.

Precondizioni

- r != NULL

Postcondizioni

- nessuna modifica alla struttura

Valore di ritorno

ValoreSignificatointmatricola della prenotazione

3. res_get_checked

Specifica sintattica

```
int res_get_checked(const Reservation* r);
```

Semantica

Restituisce lo stato di check-in della prenotazione.

Precondizioni

- `r != NULL`

Postcondizioni

- nessuna modifica alla struttura

Valore di ritorno

ValoreSignificato0prenotazione non confermata1prenotazione confermata

4. `res_set_checked`

Specifica sintattica

```
void res_set_checked(Reservation* r, int v);
```

Semantica

Modifica il valore del campo `checked`.

Precondizioni

- `r != NULL`
- `v` deve rappresentare un valore booleano (0 oppure 1)

Postcondizioni

- `r->checked = v`

Valore di ritorno

Nessuno (`void`).

5. `res_save`

Specifica sintattica

```
void res_save(const Reservation* r, FILE* f);
```

Semantica

Salva la prenotazione su file in formato testuale.

Formato scritto:

matricola data fascia posto checked

Precondizioni

- `r != NULL`
- `f != NULL`
- file aperto in modalità scrittura

Postcondizioni

- i dati della prenotazione vengono scritti nel file

Valore di ritorno

Nessuno (void).

6. `res_load`

Specifica sintattica

`Reservation* res_load(FILE* f);`

Semantica

Legge una prenotazione da file e crea dinamicamente la relativa struttura.

Precondizioni

- `f != NULL`
- file aperto in modalità lettura
- il file deve contenere dati nel formato corretto:

`int string int int int`

Postcondizioni

Se la lettura ha successo:

- viene allocata una nuova `Reservation`
- i campi vengono inizializzati con i dati letti dal file

Se la lettura fallisce:

- nessuna struttura viene creata

Valore di ritorno

ValoreSignificatopuntatore a Reservationprenotazione caricata correttamenteNULLerrore di lettura o fine file

7. res_destroy

Specifica sintattica

```
void res_destroy(Reservation* r);
```

Semantica

Libera la memoria occupata da una prenotazione.

Precondizioni

- $r \neq \text{NULL}$
- r deve essere stato allocato dinamicamente

Postcondizioni

- memoria della struttura liberata

Valore di ritorno

Nessuno (void).

Student_listc

sl_create

Specifica sintattica

```
StudentList* sl_create();
```

Specifica semantica

Crea dinamicamente una lista vuota di studenti.

Precondizioni

- Nessuna.

Postcondizioni

- Viene allocata memoria per una nuova lista.

- head == NULL.

Valore di ritorno

- Puntatore alla nuova StudentList.
- NULL in caso di fallimento di malloc (non gestito esplicitamente nel codice).

2. sl_add

Specifica sintattica

```
void sl_add(StudentList* l, Student* s);
```

Specifica semantica

Inserisce uno studente in testa alla lista.

Precondizioni

- l != NULL
- s != NULL

Postcondizioni

- Viene creato un nuovo nodo.
- Il nuovo nodo contiene s.
- Il nodo diventa la nuova testa della lista.
- Gli elementi precedenti restano collegati.

Valore di ritorno

- Nessuno (void).

3. sl_find

Specifica sintattica

```
Student* sl_find(StudentList* l, int m);
```

Specifica semantica

Cerca nella lista uno studente con matricola uguale a m.

Precondizioni

- l != NULL

Postcondizioni

- Nessuna modifica alla lista.

Valore di ritorno

- Puntatore allo studente trovato.
- NULL se nessuno studente possiede quella matricola.

4. sl_save

Specifica sintattica

```
void sl_save(StudentList* l, FILE* f);
```

Specifica semantica

Salva su file tutti gli studenti presenti nella lista.

Precondizioni

- l != NULL
- f != NULL
- Il file deve essere aperto in modalità scrittura.

Postcondizioni

- Tutti gli studenti della lista vengono scritti nel file tramite student_save().

Valore di ritorno

- Nessuno (void).

5. sl_load

Specifica sintattica

```
StudentList* sl_load(FILE* f);
```

Specifica semantica

Carica dal file una lista di studenti.

Precondizioni

- `f != NULL`
- Il file deve essere aperto in modalità lettura.

Postcondizioni

- Viene creata una nuova lista.
- Ogni studente letto tramite `student_load()` viene inserito nella lista.
- La lettura termina quando `student_load()` restituisce `NULL`.

Valore di ritorno

- Puntatore alla nuova lista caricata.

6. `sl_destroy`

Specifica sintattica

```
void sl_destroy(StudentList* l);
```

Specifica semantica

Libera tutta la memoria associata alla lista e agli studenti contenuti.

Precondizioni

- `l != NULL`

Postcondizioni

- Tutti i nodi vengono deallocati.
- Tutti gli studenti vengono distrutti tramite `student_destroy()`.
- La memoria della lista viene liberata.

Valore di ritorno

- Nessuno (`void`).

Systemc

Funzione system_load

Specifica sintattica

System* system_load();

Specifica semantica

Carica dal filesystem i dati del sistema:

- elenco studenti da studenti.txt
- prenotazioni da prenotazioni.txt

Se i file non esistono:

- crea una lista studenti vuota
- inizializza l'array prenotazioni vuoto.

Valore di ritorno

- Restituisce un puntatore a una struttura System inizializzata.
- Restituisce un sistema valido anche se i file non esistono.

Precondizioni

- Le funzioni sl_load, sl_create, res_load devono essere definite correttamente.
- I file, se presenti, devono avere formato corretto.

Postcondizioni

- s->studenti contiene gli studenti caricati.
- s->pren contiene tutte le prenotazioni caricate.
- s->n contiene il numero totale delle prenotazioni.

Funzione system_save

Specifica sintattica

void system_save(System* s);

Specifica semantica

Salva:

- gli studenti nel file studenti.txt
- le prenotazioni nel file prenotazioni.txt.

Valore di ritorno

- Nessuno (void).

Precondizioni

- `s != NULL`
- I file devono essere scrivibili.

Postcondizioni

- I dati presenti nel sistema sono memorizzati nei rispettivi file.

Funzione `system_destroy`

Specifica sintattica

```
void system_destroy(System* s);
```

Specifica semantica

Libera tutta la memoria allocata dinamicamente:

- prenotazioni
- array prenotazioni
- lista studenti
- struttura System.

Valore di ritorno

- Nessuno (void).

Precondizioni

- `s != NULL`

Postcondizioni

- Tutta la memoria associata al sistema viene liberata.

Funzione `system_register_student`

Specifica sintattica

```
void system_register_student(System* s, int m, const char* nome,  
const char* corso);
```

Specifica semantica

Registra un nuovo studente nel sistema tramite:

- matricola m
- nome
- corso di laurea.

Valore di ritorno

- Nessuno (void).

Precondizioni

- `s != NULL`
- `nome != NULL`
- `corso != NULL`
- La matricola deve essere valida.

Postcondizioni

- Lo studente viene aggiunto alla lista studenti.

Funzione `system_book`

Specifica sintattica

```
void system_book(System* s, int m, const char* data, int fascia);
```

Specifica semantica

Crea una nuova prenotazione associata:

- alla matricola m
- alla data specificata
- alla fascia oraria indicata.

La prenotazione viene aggiunta all'array dinamico del sistema.

Valore di ritorno

- Nessuno (void).

Precondizioni

- s != NULL
- data != NULL
- fascia valida.
- Lo studente deve esistere.

Postcondizioni

- La prenotazione viene inserita nel sistema.
- s->n aumenta di 1.

Funzione system_checkin

Specifica sintattica

```
void system_checkin(System* s, int m);
```

Specifica semantica

Registra nel file storico.txt l'ingresso dello studente con matricola m.

Valore di ritorno

- Nessuno (void).

Precondizioni

- s != NULL
- Il file storico.txt deve essere apribile in append.

- La matricola deve essere valida.

Postcondizioni

- Nel file storico viene aggiunta una riga:

IN matricola

Funzione system_checkout

Specifica sintattica

```
void system_checkout(System* s, int m);
```

Specifica semantica

Registra nel file storico.txt l'uscita dello studente con matricola m.

Valore di ritorno

- Nessuno (void).

Precondizioni

- $s \neq \text{NULL}$
- Il file storico.txt deve essere apribile.
- La matricola deve essere valida.

Postcondizioni

- Nel file storico viene aggiunta una riga:

OUT matricola

Funzione system_report

Specifica sintattica

```
void system_report(System* s);
```

Specifica semantica

Mostra:

- il numero totale di prenotazioni

- il numero totale di accessi effettuati leggendo storico.txt.

Conta gli accessi considerando solo le righe che iniziano con IN.

Valore di ritorno

- Nessuno (void).

Precondizioni

- $s \neq \text{NULL}$

Postcondizioni

- Le statistiche vengono stampate a video.

RELAZIONE SUI CASI DI TEST

Sono stati effettuati tutti i casi di test voluti dalla traccia e hanno portato esito positivo.

REALIZZATO DA:

STUDENTE: SANTILLO LUIGI

MATRICOLA: NF12101147

STUDENTE: SALERNO RAFFAELE

MATRICOLA: NF12100527